

First Loss: Evidence-Boundary Reasoning for Root-Cause Attribution

KuangYu Chen

Veridical Tech, Inc.

Xi'an University of Science and Technology Hi-tech College

rin@rin.red

Preprint candidate v1.1.0, September 2026

Abstract

Failures in multi-stage software systems often become visible after the evidence needed for a correct result has already disappeared or changed meaning. Debugging from the final symptom can therefore blame a component that never received the relevant evidence. First Loss Methodology (FLM) treats an execution as a sequence of evidence transformations. *First Loss* marks the first boundary where evidence becomes unavailable to the next stage; *Invariant Loss* marks the first observed violation of identity, order, scope, time, provenance, authority, or lifecycle. Neither is automatically the root cause, which still requires intervention.

We formalize this distinction, limit the attribution rule to the observed lineage segment, and define bounded answers for missing observations, explicit exceptions for nonlocal effects, and segment-aware handling of evidence reacquisition. A controlled benchmark with 1,200 test cases and a 600-case replay window checks the internal behavior of the rules. Three separate executable reference runtimes for retrieval-augmented generation, ETL, and build dependency resolution add 360 cases with partial traces, noisy probes, nonlocal side effects, and reacquisition. FLM attains 0.981 safe-attribution accuracy in this second evaluation; removing independent replay lowers it to 0.972, while removing intervention reduces confirmed root-cause accuracy from 0.961 to zero. A 300-response GLM baseline reaches 0.920 overall attribution accuracy and 0.973 exact-path root-cause accuracy, but only 0.200 exact agreement across five repeated runs. These results establish reproducibility and boundary behavior in controlled systems. They do not establish production-wide generality.

1 Introduction

A reader fails to cite a document. A build fails at the linker. A record is missing from a downstream table. The visible component is an obvious place to start looking, but it may be the wrong place to assign cause. In each example, the required evidence may have vanished earlier: a ranked document was reordered before a cap, an artifact version changed during resolution, or a join key was corrupted before the record reached the sink.

Fault localization usually ranks suspicious program locations from failing executions [6, 1]. Cause-transition and cause-effect work asks where a failing state first diverges from a passing one [12, 5]. Distributed tracing follows requests across service boundaries [10, 9], while provenance records where data came from and how it changed [2, 4]. FLM builds on these ideas but asks a narrower question: what may be blamed when a particular piece of evidence did not survive the path to the observed failure?

The method separates three locations that are easy to collapse into one:

- **Invariant Loss**: the first observed semantic corruption;
- **First Loss**: the first boundary where the evidence becomes unavailable;
- **Root cause**: the factor whose controlled change removes the failure.

This separation gives FLM its main rule: a stage cannot be the direct-path cause of losing evidence that never reached it. The rule is intentionally narrower than “the downstream component is healthy.” Shared state, callbacks, concurrency, and hidden protocols can still create nonlocal causes.

This paper contributes (1) an evidence-lineage model with explicit loss and invariant boundaries; (2) rules for exact, bounded, nonlocal, and reacquired cases; (3) a confirmation protocol based on a frozen baseline and single-variable intervention; and (4) two reproducible evaluations, including three executable reference runtimes. The artifact is archived separately [3].

2 Related work

2.1 Fault localization and cause transitions

Spectrum-based fault localization uses coverage and test outcomes to rank suspicious program entities [6, 1]. It is useful when the target is a statement, branch, or method. FLM instead follows a named evidence item across system boundaries. The two approaches can be combined: FLM can first narrow the admissible evidence path, after which fault localization can rank code within the remaining stages.

Zeller’s cause-effect chains and Cleve and Zeller’s cause transitions identify differences that connect a failure to its origin [12, 5]. FLM shares the interest in early divergence but does not equate the first semantic divergence with the first disappearance. A record may keep flowing after its tenant, ordering, or version invariant has failed.

2.2 Provenance and distributed traces

Data provenance distinguishes why an output exists from where its contributing values came from [2, 4]. FLM uses lineage as its identity backbone, then adds consumability and stage-specific invariants. Provenance alone does not say whether an output remained suitable for the next consumer.

Request-flow comparison and distributed tracing reconstruct cross-service execution paths [9, 10]. MicroRCA combines service dependencies, metrics, and anomaly information to localize performance faults [11]. FLM is not a replacement for those systems. It is an attribution discipline that can consume their traces when the incident depends on evidence survival.

2.3 Causal confirmation and RAG pipelines

Counterfactual reasoning distinguishes association from intervention [8]. FLM therefore labels a root cause *confirmed* only after a frozen fail-before/pass-after comparison and an independent replay. In retrieval-augmented generation, retrieved documents are explicit non-parametric evidence [7]; ranking, metadata enrichment, context caps, and assembly make that setting a natural example, but the method is not tied to RAG.

3 Method

3.1 Evidence transformations

Let a system instance be

$$\mathcal{M} = (\mathcal{E}, \mathcal{S}, \Omega), \tag{1}$$

where $\mathcal{E}$ is the evidence universe, $\mathcal{S} = (S_1, \dots, S_n)$ is an ordered path of transformations, and Ω is the observation model. An evidence item

$$E = (id, c, m, p, a, l) \tag{2}$$

contains identity, content, metadata, provenance, authority, and lifecycle state. A transformation may copy, split, merge, enrich, reorder, filter, or reject an item. Lineage identity is therefore a recoverable mapping, not object identity.

For a lineage representation of E after stage i , define

$$P_i(E) = \begin{cases} 1 & \text{if the representation is available and consumable downstream,} \\ 0 & \text{if it is unavailable or unusable,} \\ ? & \text{if } \Omega \text{ does not observe the boundary.} \end{cases} \quad (3)$$

Consumability matters. A byte string that survives storage but cannot be decoded by the next stage is absent for this analysis.

3.2 First Loss and Invariant Loss

Within one continuously observed lineage segment,

$$\text{FL}(E) = \min\{i : P_i(E) = 1 \wedge P_{i+1}(E) = 0\}. \quad (4)$$

The First Loss stage is S_{i+1} . If an item later returns through fallback retrieval or a side input, the return starts a new segment. FLM reports the earlier loss and the reacquisition; it does not pretend that presence is monotone.

Let $I_{i,v}^{obs}(E)$ and $I_{i,v}^{exp}(E)$ be the observed and expected values of invariant v after stage i . Then

$$\text{IL}(E) = \min\{i : \exists v, I_{i,v}^{obs}(E) \neq I_{i,v}^{exp}(E)\}. \quad (5)$$

An ordering violation can precede a cap that removes the item, so $\text{IL}(E) < \text{FL}(E)$. An expired item may remain present until the final output, in which case Invariant Loss exists without an earlier disappearance.

3.3 Attribution boundary and its limits

Suppose $\text{FL}(E) = k$ on an observed lineage segment and the failure propagates through that same evidence path. Stages after S_k are excluded as *direct-path causes* of losing E . This is the attribution boundary. It does not exclude four alternatives:

1. a later stage has an independent defect;
2. an upstream stage changes shared state used downstream;
3. an implicit callback or concurrent action bypasses the observed path;
4. the evidence is reacquired from another source.

When one of these mechanisms is plausible, FLM returns a cross-stage exception instead of a component-wide exclusion.

If $P_i(E) = 1$, $P_j(E) = 0$, and one or more boundaries between i and j are unobserved, the method reports the interval $(S_i, S_j]$. It must not select a hidden stage merely because that stage is expected from the architecture. This is *bounded attribution*.

3.4 Confirmation protocol

FLM distinguishes a candidate from a confirmed cause. Confirmation requires a reproducible failing baseline, a recorded First Loss or bounded interval, an explicit invariant or rejection predicate, a minimal counterexample, a single-variable intervention, fail-before/pass-after evidence, upstream checks, and an independent replay. Without intervention, the output remains a candidate set. Without replay, transient probe errors remain unresolved.

The confirmation level is also granular. An intervention that changes an entire metadata stage confirms a stage-level cause. Predicate-level confirmation needs the exact configuration, code revision, and before/after trace for the predicate itself.

Table 1: Controlled benchmark, 1,200 test cases.

Method	FLA	ILA	RCA	Confirmed	1-FDAR
Final symptom	.150	.450	.050	.050	.150
Component RCA	1.000	1.000	.420	.420	.578
Existence only	1.000	.450	.500	.500	1.000
Trace pattern	1.000	1.000	1.000	1.000	1.000
FLM	1.000	1.000	1.000	1.000	1.000

Table 2: Executable-runtime evaluation, 360 cases.

Method	Safe	FLA	RCA	Confirmed	Overclaim
Final symptom	.000	.000	.000	.000	1.000
Component heuristic	.000	.944	.000	.000	1.000
FLM, no intervention	.981	.961	.961	.000	.000
FLM, no replay	.972	.944	.944	.944	.000
FLM	.981	.961	.961	.961	.000

4 Evaluation

We ask four questions: whether FLM locates evidence loss, whether it separates semantic corruption from disappearance, whether it avoids unsafe downstream attribution, and whether intervention and replay change the strength of the result.

4.1 Controlled rule benchmark

The first benchmark freezes 1,200 test cases and 600 cases in a separate seed window. It covers 16 single-fault and four compound-fault families over one nine-stage abstract pipeline. Ground truth is generated from the injection specification before any method runs. This benchmark checks rule implementation and reproducibility, not domain generality.

Table 1 reports the central result. A deterministic trace-pattern baseline matches FLM exactly, which is expected because both receive complete traces. The meaningful comparison is with methods that omit the attribution boundary or invariant checks.

The published run is exactly reproducible. We also corrected the bootstrap implementation in v1.1.0 so paired comparisons resample shared case indices rather than drawing each method independently. Point estimates are unchanged; confidence intervals are regenerated from the corrected procedure.

4.2 Executable heterogeneous runtimes

The second benchmark does not rename one common pipeline. It executes three separate implementations:

- a RAG path that ranks documents, enriches metadata, applies fallback retrieval, caps context, and answers;
- an ETL path that parses events, maps keys, replays dead letters, joins dimensions, and commits a window;
- a build path that resolves versions, reads a cache, falls back to vendored artifacts, links, and tests.

Each runtime has six profiles: local invariant-first failure, direct loss, hidden loss-stage observation, noisy probes, an explicit nonlocal side effect, and evidence reacquisition. Twenty seeded repetitions per runtime/profile yield 360 cases. Five probes are available in noisy cases.

Safe attribution requires an exact answer for fully observed local paths, a containing interval for hidden observations, an explicit exception for nonlocal effects, and segment-aware handling of reacquisition. Table 2 reports the results.

Table 3: Real-model baseline, 60 cases with five runs each.

Metric	Result
Attribution accuracy	.920
Exact-path FLA / ILA / RCA	.987 / .813 / .973
Status accuracy	.927
Risky-case overclaim rate	.047
Five-run exact agreement	.200
Final schema adherence	1.000

The intervention ablation now measures what the protocol claims: localization can remain correct, but the result is not confirmed. Replay helps under probe noise, though five repetitions do not eliminate every error. The nonlocal profile provides an explicit side-effect signal, so it tests exception handling rather than discovery of hidden nonlocal causes.

4.3 Real-LLM baseline

The real-model baseline draws 60 cases stratified by runtime and profile and repeats each case five times. It hides profile and ground truth, fixes the prompt and output contract, and records the requested and provider-reported model identifiers, temperature, prompt hash, response identifier, usage, latency, raw output, and per-run scores. The model does not receive the FLM boundary rule.

We requested `glm-5.3-flash` through an OpenAI-compatible Responses endpoint; the provider reported `glm-5-3-flash`. The gateway accepted but did not enforce strict JSON Schema, so two compatibility pilots were excluded. We then froze prompt version `llm-rca-v1.1.1`, used JSON-object mode with exact field validation, and ran 300 requests. The final run consumed 218,555 input and 90,129 output tokens.

The model’s exact-path RCA is slightly higher than FLM’s 0.961 on this sample. FLM has the stronger overall controlled result because it explicitly returns bounded, exception, and reacquisition states, but the comparison is not a model leaderboard: the methods have different confirmation semantics, the sample is small, and the gateway’s model build is not independently verifiable. The low five-run agreement is the clearest model result. A majority-vote analysis should be preregistered before it is used as a headline metric.

5 Production incident replay

The motivating incident occurred in the SPM-Polaris recall pipeline. Vector retrieval returned ranked evidence. Metadata enrichment preserved the records but not their order. A downstream cap then removed the target by position. The target never reached selection, gating, assembly, or the reader.

The observed Invariant Loss was the metadata stage, and First Loss occurred at the cap. Preserving rank across metadata lookup, with the query, corpus, retrieval parameters, and cap fixed, removed the failure in another process and time window. The released replay supports a confirmed *stage-level* cause at metadata enrichment. Because the public artifact omits the production query, corpus revision, code revision, and before/after payloads, the specific predicate remains a candidate rather than a publicly reproducible confirmed cause.

6 Threats to validity

Construct validity. The first benchmark encodes FLM-compatible ground truth, so its perfect result is an implementation check. The executable runtimes reduce that coupling but remain authored reference systems. Safe attribution is a method-specific outcome and should not replace end-to-end incident resolution time.

Internal validity. Faults and probes are controlled, and the same authors designed the method and evaluation. The final real-LLM prompt was frozen after two compatibility pilots, not before any contact with the endpoint. Those pilots are excluded and documented, but prompt adaptation can bias the result. The

gateway’s provider-reported model ID is not a versioned model snapshot. A preregistered external evaluation would reduce both risks.

External validity. Three executable runtimes are more informative than seven renamed instances, but they are small and deterministic. The study does not cover production concurrency, organizational handoffs, or unknown ground truth. One production replay shows plausibility, not prevalence.

Conclusion validity. Confidence intervals and McNemar tests describe repeated benchmark cases, not a random sample of all incidents. Multiple baseline comparisons should be treated as secondary. Very small exact p -values are reported as bounds rather than numerical zero.

7 Discussion

FLM is most useful when evidence has a stable or recoverable identity, stages are ordered along a relevant path, invariants can be checked, and at least a few boundaries can be observed or replayed. When those conditions fail, the honest output is an interval or an exception. That is not a failed diagnosis. It is the boundary of what the trace supports.

The method also changes the order of work. Engineers first ask whether the evidence reached a stage, then whether it still meant the same thing, and only then whether that stage behaved correctly. This ordering does not make causal reasoning automatic, but it stops a common mistake: investigating a component as the cause of losing input it never received.

8 Conclusion

First Loss is an attribution boundary, not a root cause. Invariant Loss can occur earlier, and intervention supplies the causal claim that traces alone cannot. By limiting exclusion to an observed lineage segment, reporting bounded intervals, and recognizing nonlocal and reacquired paths, FLM turns a debugging intuition into a rule that can fail safely. The current evidence supports reproducibility and controlled boundary behavior. Production generality remains an empirical question.

Artifact and disclosures

Code is licensed under MIT. Benchmark data and paper text are licensed under CC BY 4.0. The author designed the method, validated the experiments, and approved the manuscript. AI assistants supported prose drafting, code implementation, and experiment organization. No human-subject experiment was conducted. No external funding is reported.

References

- [1] Rui Abreu, Peter Zoetewij, Rob Golsteijn, and Arjan J. C. van Gemund. A practical evaluation of spectrum-based fault localization. *Journal of Systems and Software*, 82(11):1780–1792, 2009.
- [2] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory – ICDT 2001*, pages 316–330. Springer, 2001.
- [3] KuangYu Chen. First loss methodology: Benchmark and reproducibility artifact, 2026.
- [4] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009.
- [5] Holger Cleve and Andreas Zeller. Locating causes of program failures. In *Proceedings of the 27th International Conference on Software Engineering*, pages 342–351, 2005.
- [6] James A. Jones, Mary Jean Harrold, and John Stasko. Visualization of test information to assist fault localization. In *Proceedings of the 24th International Conference on Software Engineering*, pages 467–477, 2002.

- [7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [8] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2 edition, 2009.
- [9] Raja R. Sambasivan, Alice X. Zheng, Michael De Rosa, Elie Krevat, Spencer Whitman, Michael Stroucken, William Wang, Lianghong Xu, and Gregory R. Ganger. Diagnosing performance changes by comparing request flows. In *8th USENIX Symposium on Networked Systems Design and Implementation*, pages 43–56. USENIX Association, 2011.
- [10] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspán, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. Technical report, Google, 2010.
- [11] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. MicroRCA: Root cause localization of performance issues in microservices. In *2020 IEEE/IFIP Network Operations and Management Symposium*, pages 1–9, 2020.
- [12] Andreas Zeller. Isolating cause-effect chains from computer programs. In *Proceedings of the 10th ACM SIGSOFT Symposium on Foundations of Software Engineering*, pages 1–10, 2002.